R7023E OBDH Assignment - Function Trees

Sheets

FT_Ground - hierarchical function tree for the Ground Segment (MCS).
FT_Space - hierarchical function tree for the Space Segment: Platform OBC (PF) and Payload ICU (PL).
Mode_Transitions - mode transition tables for Ground, PF, onboard time and PL.
TC_TM_List - TC/TM definitions derived from the function tree; becomes the MIB text file.
Requirements - assignment requirements R01-R32; 'Functions tracing' counts how many FT functions cover each.

How to read the function trees

Level 1 = SW element, Level 2 = function group, Level 3 = function (leaf).
Rule from the assignment: 1 function definition -> 1 code item -> 1 test. Each leaf has a proposed Python code item and a test ID.
Trigger: Init / Boot = once at start; Cyclic = in main loop; TC / Per TM / Event = when that occurs.
Traces to: requirement IDs on the Requirements sheet (basis for the SDD traceability matrix).

Assumed deployment

Three Docker containers (ground, obc, and payload).
Code item names, mode durations (10 s / 5 s) and the HK period are proposals

ID	Level 1	Level 2	Level 3 (function)	Description	Trigger	Code item	Test ID	Traces to
G	**Ground MCS**			**Ground Mission Control System process (Docker container 'ground'). Only element with user interaction.**		**ground/main.py**		**R01**
G1		**Initialisation**		**Bring the ground process up into a usable state.**				
G1.1			Load MIB	Read MIB text file; build table of valid TCs (name, target, args, ranges, time-tag allowed) and TMs.	Init	mib.load_mib()	T-G1.1	R05
G1.2			Open HK log file	Open/create append-mode logfile for received HK TM.	Init	tmlog.open_log()	T-G1.2	R04
G1.3			Connect to OBC	Open TCP socket to the OBC container (space link). Retry until connected.	Init / link loss	link.connect()	T-G1.3	R01
G1.4			Ground time base	Use container/host local wall clock as ground time (TZ must match OBC container).	Init	gtime.now()	T-G1.4	R17
G2		**Telecommand handling**		**Turn operator text into a verified, sequence-counted TC and uplink it.**				
G2.1			Read operator input	Plain text prompt; read one TC line (e.g. 'SET-PL-POWER 1 @14:05:00').	Cyclic (console)	console.read_cmd()	T-G2.1	R02
G2.2			Parse TC	Split line into command name, arguments and optional time tag.	Operator input	tc.parse()	T-G2.2	R02, R14
G2.3			Validate TC against MIB	Unknown command / wrong arg count / out-of-range arg -> reject on ground with message, do not uplink.	Operator input	tc.validate()	T-G2.3	R05, R12
G2.4			Validate time-tag format	Check time tag is hh:mm:ss; warn if already in the past (final check is on board).	Operator input	tc.check_timetag()	T-G2.4	R14, R27
G2.5			Identify TC target	Look up target (PF or PL) from MIB and put it into the TC.	Operator input	tc.set_target()	T-G2.5	R22
G2.6			Assign TC sequence count	Increment and attach ground TC sequence counter.	Per TC	tc.next_seq()	T-G2.6	R23
G2.7			Encode and uplink TC	Serialise TC to bit array and send to OBC.	Per TC	link.send_tc()	T-G2.7	R01
G2.8			Track pending TCs	Keep sent TCs until ACK/NACK received; flag TCs without response after timeout.	Cyclic	tc.track_pending()	T-G2.8	R13, R23
G3		**Telemetry handling**		**Receive, check, display and archive TM from space.**				
G3.1			Receive TM	Non-blocking read (thread or select) of TM packets from OBC link.	Cyclic	link.recv_tm()	T-G3.1	R03, R32
G3.2			Decode TM	Deserialise TM; identify type: HK-PF, HK-PL, EVENT, ACK, NACK.	Per TM	tm.decode()	T-G3.2	R24
G3.3			Check TM sequence count	Compare with last TM seq; report gaps / duplicates to operator.	Per TM	tm.check_seq()	T-G3.3	R23
G3.4			Correlate ACK/NACK	Match ACK/NACK to pending TC by TC seq; close or mark failed.	Per TM	tm.correlate()	T-G3.4	R12, R13
G3.5			Add ground reception timestamp	Append ground receive time (hh:mm:ss) to every TM.	Per TM	tm.stamp()	T-G3.5	R04
G3.6			Display TM	Print TM as plain text in terminal as soon as received (e.g. 'payload state = POWERED').	Per TM	console.show_tm()	T-G3.6	R03
G3.7			Store HK in logfile	After display, append HK TM (with reception timestamp) to logfile.	Per HK TM	tmlog.write()	T-G3.7	R04
G4		**Ground mode management**		**Ground element states (see Mode_Transitions).**				
G4.1			Link state handling	DISCONNECTED <-> CONNECTED; detect link loss and reconnect.	Event	link.state()	T-G4.1	R32
G4.2			Shutdown	Operator 'QUIT': close link, flush/close logfile, exit.	Operator input	main.shutdown()	T-G4.2	
G5		**Main loop**		**Cyclic loop: poll console + poll link + pending-TC timeouts.**	**Cyclic**	**main.run()**	**T-G5**	**R32**

ID	Level 1	Level 2	Level 3 (function)	Description	Trigger	Code item	Test ID	Traces to
PF	**Platform OBC**			**Platform onboard computer process (Docker container 'obc').**		**obc/main.py**		**R01**
PF1		**Boot / initialisation**		**Start OBC software into a defined safe state.**				
PF1.1			Start SCET	Start Spacecraft Elapsed Time at 00:00:00, upcounting.	Boot	obt.start_scet()	T-PF1.1	R25
PF1.2			Enter SAFE mode	Platform mode = SAFE at boot; PL power = OFF.	Boot	modes.init()	T-PF1.2	R06
PF1.3			Open Gnd link	Open TCP server for uplink/downlink to ground.	Boot	gndlink.listen()	T-PF1.3	R01
PF1.4			Open PL link	Create the Payload object	Boot	pllink.connect()	T-PF1.4	R01
PF1.5			Boot event TM	Send event TM 'OBC BOOTED, mode SAFE, time SCET'.	Boot	tm.event()	T-PF1.5	R24, R29
PF2		**Onboard time management**		**SCET / OBT handling.**				
PF2.1			Maintain onboard clock	Return SCET (elapsed) until OBT set, then OBT (wall clock based), format hh:mm:ss.	Cyclic	obt.now()	T-PF2.1	R25, R27
PF2.2			Execute SET-OBT	Set OBT from TC argument, set flag time_set=True, event TM 'OBT SET'.	TC	obt.set_obt()	T-PF2.2	R26, R17
PF2.3			Distribute OBT to PL	Call Payload.sync_timer() with the new OBT	TC / cyclic	pllink.sync_time()	T-PF2.3	R31
PF3		**TC reception and verification**		**Acceptance checks on every uplinked TC.**				
PF3.1			Receive TC	Cyclically poll ground link for uplinked TCs.	Cyclic	gndlink.poll()	T-PF3.1	R32
PF3.2			Decode TC	Deserialise TC; malformed -> NACK.	Per TC	tc.decode()	T-PF3.2	R12
PF3.3			Check TC sequence count	Detect gaps / duplicates in TC seq; report in NACK or event TM.	Per TC	tc.check_seq()	T-PF3.3	R23
PF3.4			Check command validity	Unknown command or invalid args -> NACK with reason.	Per TC	tc.check_syntax()	T-PF3.4	R12
PF3.5			Check mode dependency	PL commands rejected in SAFE / during transition; TAKE-IMAGE rejected if PL not ON.	Per TC	tc.check_mode()	T-PF3.5	R06, R08
PF3.6			Check time tag	Reject time tag already overdue vs OBT; reject time-tagged TC before OBT is set.	Per TC	tc.check_timetag()	T-PF3.6	R15
PF3.7			Send ACK / NACK	Acceptance ACK (with TC seq) or rejection NACK (with TC seq + reason) to ground.	Per TC	tm.ack() / tm.nack()	T-PF3.7	R12, R13
PF3.8			Route TC	Immediate PF TC -> execute; immediate PL TC -> PF6; time-tagged ->	Per TC	tc.route()	T-PF3.8	R22, R14
PF4		**Time-tag scheduler**		**Onboard queue of time-tagged TCs.**				
PF4.1			Insert into queue	Store TC sorted by execution time.	TC	sched.insert()	T-PF4.1	R14
PF4.2			Check queue for due TCs	Each cycle compare head of queue with OBT.	Cyclic	sched.tick()	T-PF4.2	R16, R32
PF4.3			Release and execute due TC	Re-check mode dependency at execution time, execute or report	Time due	sched.release()	T-PF4.3	R16
PF4.4			Report execution	Event TM 'TT TC <seq> executed / failed at <OBT>'.	Time due	tm.event()	T-PF4.4	R16, R29
PF5		**Platform mode**		**SAFE / NADIR / TARGET-POINTING with timed transitions.**				
PF5.1			Hold platform mode	Current mode + target mode + transition end time.	Cyclic	modes.state	T-PF5.1	R06, R07
PF5.2			Start mode transition	Execute SET-PF-MODE: enter TRANSITION, start timer (e.g. 10 s).	TC	modes.request()	T-PF5.2	R07, R10
PF5.3			Complete mode transition	When timer expires set new mode and send event TM 'MODE <x>	Cyclic	modes.update()	T-PF5.3	R10, R11, R29
PF5.4			Automatic PL shutdown on	On any transition to SAFE command PL OFF first, then event TM.	Event	modes.enter_safe()	T-PF5.4	R09
PF5.5			Reject invalid transitions	E.g. same mode, mode change while in transition -> NACK.	TC	modes.check()	T-PF5.5	R12
PF6		**Payload interface (OBC**		**Everything the OBC does towards the PL ICU.**				
PF6.1			Simulate PL power switch	Hold PL power parameter (0/1) in OBC; switching notifies PL ICU.	TC	plif.set_power()	T-PF6.1	R18
PF6.2			Forward PL TC to ICU	Send PL command (with OBT) to PL; wait for PL response.	TC	plif.send_cmd()	T-PF6.2	R08, R22
PF6.3			Receive PL responses	Read PL replies / image start & finish reports.	Cyclic	plif.poll()	T-PF6.3	R19
PF6.4			Request PL HK	Cyclically (and on ground request) ask PL ICU for its HK.	Cyclic / TC	plif.request_hk()	T-PF6.4	R24

ID	Level 1	Level 2	Level 3 (function)	Description	Trigger	Code item	Test ID	Traces to
PF6.5			Forward PL HK to ground	Wrap PL data (incl. image start/finish) into HK-PL TM and downlink.	Per PL TM	tm.hk_pl()	T-PF6.5	R20
PF6.6			PL ON/OFF event TM	Event TM confirming PL ON / OFF execution with OBT.	Event	tm.event()	T-PF6.6	R21, R29
PF7		**Telemetry generation**		**Cyclic HK, event TM, TM on request.**				
PF7.1			Cyclic PF HK	Every N s: OBT/SCET, time_set, PF mode, PL power, TC counters,	Cyclic	tm.hk_pf()	T-PF7.1	R24, R28
PF7.2			HK on request	REQ-HK TC: send PF HK immediately.	TC	tm.hk_pf()	T-PF7.2	R24
PF7.3			Event TM generation	Event packets for mode changes, PL switching, OBT set, TT execution,	Event	tm.event()	T-PF7.3	R24, R29
PF7.4			Assign TM sequence count	Increment TM counter for each downlinked packet.	Per TM	tm.next_seq()	T-PF7.4	R23
PF7.5			Encode and downlink TM	Serialise TM and send to ground.	Per TM	gndlink.send_tm()	T-PF7.5	R01
PF8		**OBC main loop**		**Cyclic loop (e.g. 10 Hz): poll uplink, poll PL, tick clock, mode**	**Cyclic**	**main.run()**	**T-PF8**	**R32**
PL	**Payload ICU**			**Payload instrument control unit process (Docker container**		**payload/main.py**		**R01**
PL1		**Initialisation**						
PL1.1			Start in OFF	PL mode = OFF at start.	Boot	plmodes.init()	T-PL1.1	R18
PL1.2			Open image log	Open/create image take log text file.	Boot	imglog.open()	T-PL1.3	R30
PL2		**PL time**						
PL2.1			Receive OBT from OBC	Store last OBT + local offset from sync message / command; compute	Per message	pltime.update()	T-PL2.1	R31
PL3		**Command execution**						
PL3.1			Receive PL command	Cyclically poll OBC link.	Cyclic	obclink.poll()	T-PL3.1	R32
PL3.2			Validate in PL mode	E.g. TAKE-IMAGE only when ON (not OFF, not already IMAGING) ->	Per command	plcmd.check()	T-PL3.2	R12
PL3.3			Execute power ON / OFF	Switch PL mode OFF <-> ON; reply with status.	Command	plmodes.set_power()	T-PL3.3	R18
PL4		**Image take**		**Simulated data generation.**				
PL4.1			Start image take	Mode -> IMAGING, report 'IMAGE START at <OBT>' to OBC.	Command	imaging.start()	T-PL4.1	R19
PL4.2			Write image log entry	Append take time (OBT) + image no. to text file.	Command	imglog.write()	T-PL4.2	R30
PL4.3			Finish image take	After 5 s: mode -> ON, report 'IMAGE END at <OBT>' to OBC.	Cyclic (timer)	imaging.update()	T-PL4.3	R19
PL5		**PL housekeeping**						
PL5.1			Provide PL HK	On OBC request reply with PL mode, images taken, last image time.	Request	plhk.report()	T-PL5.1	R20, R24
PL6		**PL main loop**		**Cyclic loop: poll OBC link, update imaging timer.**	**Cyclic**	**main.run()**	**T-PL6**	**R32**

Element	From state	Trigger (TC / event)	Guard (condition)	To state	Duration	On-board actions	TM to ground
Ground	INIT	Start	MIB loaded, logfile open	DISCONNECTED	-	-	-
Ground	DISCONNECTED	Link established	-	CONNECTED	-	Enable TC uplink	-
Ground	CONNECTED	Link lost	-	DISCONNECTED	-	Block uplink, retry connect	-
Ground	any	Operator QUIT	-	SHUTDOWN	-	Close link, close logfile	-
PF OBC	BOOT	Process start	-	SAFE	-	Start SCET, PL power OFF	EVT: BOOT, SAFE
PF OBC	SAFE	SET-PF-MODE NADIR / TARGET	OBT set (team choice)	TRANSITION	10 s	Start transition timer	ACK
PF OBC	TRANSITION	Timer expired	-	NADIR / TARGET	-	Set mode	EVT: MODE <x> ACQUIRED
PF OBC	NADIR	SET-PF-MODE TARGET	-	TRANSITION	10 s	Start timer	ACK
PF OBC	TARGET	SET-PF-MODE NADIR	-	TRANSITION	10 s	Start timer	ACK
PF OBC	NADIR / TARGET	SET-PF-MODE SAFE	-	TRANSITION (to SAFE)	5 s	PL power OFF first, abort imaging	EVT: PL OFF; EVT: MODE SAFE ACQUIRED
PF OBC	TRANSITION	Any SET-PF-MODE / PL TC	-	(unchanged)	-	Reject	NACK: in transition
PF OBC	SAFE	Any PL TC	-	(unchanged)	-	Reject	NACK: PF in SAFE
Onboard time	SCET	SET-OBT hh:mm:ss	Valid time format	OBT	-	Set clock, sync PL	EVT: OBT SET
Onboard time	SCET	Time-tagged TC	-	(unchanged)	-	Reject	NACK: OBT not set
Payload	OFF	SET-PL-POWER 1	PF in NADIR / TARGET	ON	-	Power on	EVT: PL ON
Payload	ON	PL-TAKE-IMAGE	-	IMAGING	5 s	Log image with OBT	HK-PL: IMAGE START
Payload	IMAGING	Timer expired	-	ON	-	-	HK-PL: IMAGE END
Payload	ON	SET-PL-POWER 0	-	OFF	-	Power off	EVT: PL OFF
Payload	IMAGING	SET-PL-POWER 0 / PF -> SAFE	-	OFF	-	Abort image (logged)	EVT: PL OFF
Payload	OFF	PL-TAKE-IMAGE	-	(unchanged)	-	Reject	NACK: PL OFF

Kind	Name	Target / direction	Arguments	Time-tag allowed	Mode constraint	Response / note
TC	SET-OBT	PF	hh:mm:ss	No	-	ACK + EVT OBT SET
TC	REQ-HK	PF	-	Yes	-	ACK + HK-PF
TC	SET-PF-MODE	PF	SAFE | NADIR | TARGET	Yes	Not in TRANSITION	ACK + EVT MODE ACQUIRED
TC	SET-PL-POWER	PL	0 | 1	Yes	PF in NADIR/TARGET	ACK + EVT PL ON/OFF
TC	PL-TAKE-IMAGE	PL	-	Yes	PF not SAFE, PL ON	ACK + HK-PL START/END
TC	REQ-PL-HK	PL	-	Yes	PL ON	ACK + HK-PL
TM	ACK	Gnd	tc_seq, cmd	-	-	Acceptance of TC
TM	NACK	Gnd	tc_seq, cmd, reason	-	-	Rejection of TC
TM	HK-PF	Gnd	obt, time_set, pf_mode, pl_power, tc_rx, tc_rej, tt_queue	-	-	Cyclic every N s / on request
TM	HK-PL	Gnd	obt, pl_mode, images, last_image, image_status	-	-	Cyclic / on change / on request
TM	EVENT	Gnd	obt, event_id, text	-	-	Mode change, PL on/off, OBT set, TT exec
IF	PL-CMD	OBC->PL	cmd, args, obt	-	-	Internal OBC-PL interface
IF	PL-RESP	PL->OBC	cmd, status, obt	-	-	Internal OBC-PL interface
IF	PL-HK-REQ / PL-HK	OBC<->PL	pl_mode, images, last_image	-	-	Internal OBC-PL interface

Example operator input: SET-PL-POWER 1 | PL-TAKE-IMAGE @14:05:00 (time tag optional, hh:mm:ss)

Req ID	Requirement (from assignment text)	Functions tracing	Covered?
R01	Three separate interacting SW programs: Gnd, PF, PL	8	Yes
R02	Gnd: plain text prompt to send text telecommands	2	Yes
R03	Gnd: received TM displayed in plain text as received	2	Yes
R04	Gnd: HK TM appended with ground reception timestamp, then stored in logfile	3	Yes
R05	MIB realised as text file holding TC/TM definitions and arguments	2	Yes
R06	PF SAFE mode: PL commands rejected, PL must be OFF	3	Yes
R07	PF has at least one non-SAFE mode (e.g. TARGET-Pointing)	2	Yes
R08	PL activatable/commandable only in non-SAFE PF mode	2	Yes
R09	Switch back to SAFE implies automatic PL shutdown	1	Yes
R10	PF mode transitions take time	2	Yes
R11	PF mode acquisition confirmed to Gnd by event TM	1	Yes
R12	Invalid / mistyped commands rejected with a reply	7	Yes
R13	Accepted commands acknowledged back to Gnd	3	Yes
R14	PF supports time-tagged TCs	4	Yes
R15	Time-tagged TC with overdue time tag rejected	1	Yes
R16	Time-tagged TCs executed when time tag is due	3	Yes
R17	Gnd and PF on same time base (local time)	2	Yes
R18	PL modes at least OFF and ON (simulated parameter)	3	Yes
R19	PL confirms image start and finish (~5 s) to PF	3	Yes
R20	PF forwards PL image info to Gnd in PL HK TM	2	Yes
R21	PF confirms PL ON/OFF execution to Gnd in event TM	1	Yes
R22	Command type identification (target PF or PL)	3	Yes
R23	Sequence counting of TCs and TMs	5	Yes
R24	Cyclic HK TM distinct from event TM	7	Yes
R25	OBT reported as upcounting SCET hh:mm:ss until set	2	Yes
R26	SET-OBT TC sets onboard time	1	Yes
R27	Time format hh:mm:ss in TM and TC time tags	2	Yes

Req ID	Requirement (from assignment text)	Functions tracing	Covered?
R28	Onboard time reported periodically in HK TM	1	Yes
R29	Event TM reports time of occurrence (OBT)	5	Yes
R30	PL writes image take log entry with take time to text file	2	Yes
R31	PL knows actual OBT for the image log	2	Yes
R32	SW loops cyclically; PF cyclically polls for uplinked TCs	8	Yes